

Sistemas de Control de Versiones

Instalación y uso de entornos de desarrollo

Control de Versiones

El control de versiones es un sistema que registra los cambios realizados sobre un archivo o conjunto de archivos a lo largo del tiempo.

De modo que puedas recuperar versiones específicas más adelante.

Se usan para:

- Registrar cualquier tipo de archivos
- Comparar cambios a lo largo del tiempo
- Revertir a un estado anterior del proyecto
- Ver en qué momento se introdujo un error

Control de Versiones

Dos tipos de arquitectura para clasificar los distintos SCVs:



- Arquitectura Centralizada
- Arquitectura Distribuida

Arquitectura Centralizada

Los desarrolladores usan un repositorio central al que acceden mediante un cliente en su máquina.

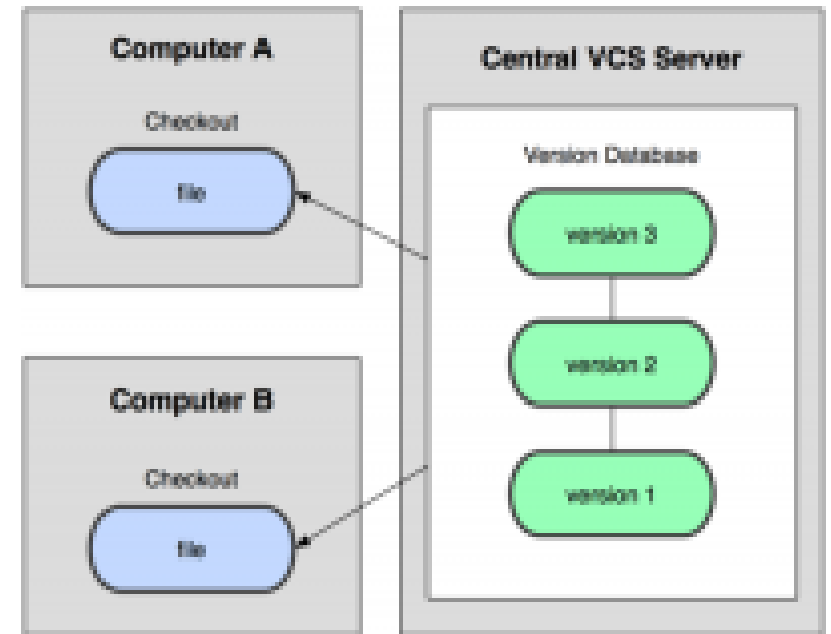
- Durante muchos años éste fue el estándar para el control de versiones.

La desventaja más obvia es el servidor centralizado como único punto de fallo.

- Si ese servidor se cae o pierde los datos, se pierde el trabajo.

Ejemplos

- Subversion SVN (Código Abierto)
- Visual SourceSafe (Propietario)



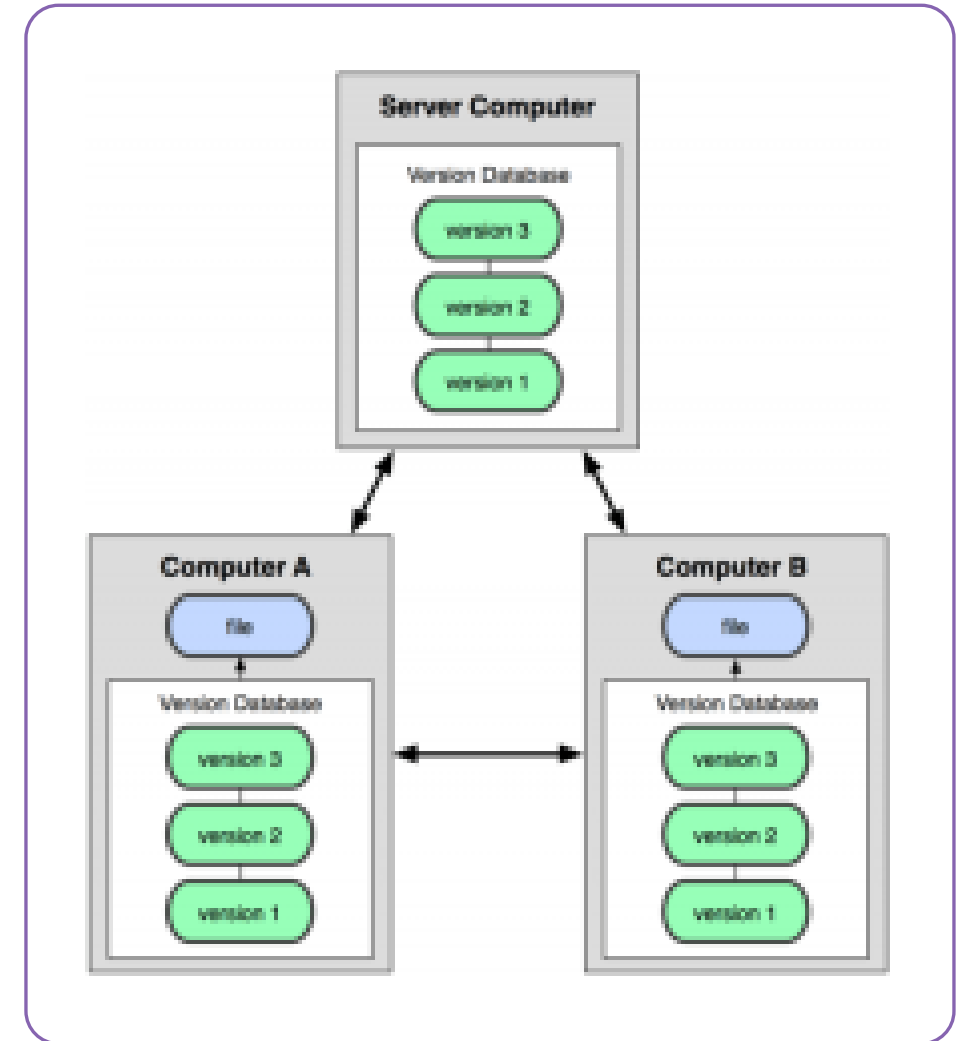
Arquitectura Distribuida

En un SCV distribuido los clientes no sólo descargan la última instantánea de los archivos.

- Cada vez que se descarga una instantánea (versión del proyecto), en realidad se hace una copia de seguridad completa de todos los datos
- Así, si un servidor muere, cualquiera de los repositorios de los clientes puede copiarse en el servidor para restaurarlo.

Ejemplos:

- GIT (Codigo Abierto)
- Mercurial (Codigo Abierto)



GIT

Es un SCV distribuido creado como necesidad durante el desarrollo de núcleo de Linux. El creador es Linus Torvalds, creador también del núcleo de Linux.

Servidores de Repositorios

Para almacenar las versiones de nuestro código necesitamos un servidor de control de versiones basado en Git.

Actualmente hay distintas opciones.

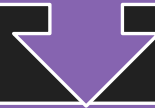
Las versiones gratuitas de los siguientes servicios ofrecen:

- GitHub: repositorios públicos ilimitados.
- BitBucket: repositorios públicos y privados ilimitados.
- GitLab: repositorios públicos y privados ilimitados.

Todos los servicios funcionan de forma similar y permiten crear una cuenta de usuario gratuita.

Funcionamiento de Git

Git guarda una copia completa del estado de los ficheros de nuestro proyecto en cada versión (commit).



Esta información se almacena en la base de datos local (repositorio local)



Posteriormente se puede sincronizar con otros repositorios remotos.

Funcionamiento de Git

En Git, los ficheros tienen 2 estados principales:

- Ficheros fuera de seguimiento (**untracked**)
- Ficheros bajo seguimiento (**tracked**)
- Ficheros **Untracked**. Cuando creo o añado ficheros nuevos a mi repositorio local estos están fuera de seguimiento. También están fuera de seguimiento los que no estaban bajo seguimiento en la última versión; último commit (confirmación).

Funcionamiento de Git

Los ficheros en seguimiento pueden tener 3 estados

Ficheros Staged (Preparados para confirmación) :

- Cuando añado ficheros untracked mediante el comando para que formen parte de la siguiente confirmación, estos pasan a estar preparados.

Ficheros unmodified (Sin modificar):

- Justo despues de hacer un commit , todos los ficheros que estaban preparados, se almacenan en la base de datos local de Git. En ese momento todos los ficheros no tienen ninguna modificación desde la última versión.

Ficheros modified (Modificados):

- Si modifico un fichero unmodified, este pasará a estado modified indicando que debo de volver a añadirlo parar prepararlo para la siguiente confirmación.

Todo este trabajo se realiza en el repositorio local y se almacena en la base de datos local de Git.

Funcionamiento de Git

